

LAPORAN TUGAS AKHIR

PEMROGRAMAN BERBASIS OBJEK KELAS D

Dosen Pengampu :

Ir. Kartono Pinaryanto, S.T., M.Cs.



NAMA :	NIM :
Natalia Margareta	255314093
Benedictus Shindo Yasashi Surya Bharata	255314100
Edgar Hansel Dharmawan	255314109
Grandinouris Tony Paguire Hallan	255314111

PROGRAM STUDI INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS SANATA DHARMA

YOGYAKARTA

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmat-Nya, sehingga kami dapat menyelesaikan laporan Tugas Akhir mata kuliah Pemrograman Berbasis Objek (PBO) ini tepat pada waktunya. Proyek aplikasi Matching Game berbasis console ini kami rancang bukan hanya sekadar untuk memenuhi syarat akademis di kelas D, melainkan juga sebagai sarana bagi kami untuk menantang diri dalam menerapkan teori pemrograman ke dalam bentuk produk nyata.

Dalam proses perkuliahan hingga penyusunan laporan ini, kami menerima banyak sekali bantuan, ilmu, dan dukungan dari berbagai pihak. Oleh karena itu, kami ingin menyampaikan terima kasih yang tulus kepada Bapak Ir. Kartono Pinaryanto, S.T., M.Cs., selaku dosen pengampu mata kuliah PBO Kelas D, yang telah memberikan bimbingan, arahan, dan pondasi logika pemrograman yang sangat berharga selama satu semester ini. Ucapan terima kasih juga kami tujukan kepada sesama anggota kelompok atas kerja sama tim yang solid, diskusi yang seru, serta komitmennya dalam memecahkan setiap error dan bug bersama-sama, tidak lupa untuk seluruh teman-teman seangkatan di program studi Informatika Universitas Sanata Dharma yang saling menyemangati selama masa perkuliahan.

Kami menyadari bahwa aplikasi maupun laporan tugas akhir ini masih jauh dari kata sempurna, dan masih banyak ruang pengembangan yang bisa digali lebih dalam lagi. Oleh karena itu, kritik dan saran yang membangun dari pembaca sangat kami harapkan. Akhir kata, semoga laporan ini dapat memberikan manfaat, wawasan baru, atau setidaknya menjadi referensi yang menarik bagi pembaca yang sedang mempelajari konsep Pemrograman Berbasis Objek

Yogyakarta, 31 Mei 2026

Tim Penyusun

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB 1 PENDAHULUAN	4
1. Latar belakang	4
2. Tujuan.....	5
BAB 2 SISTEM DAN DIAGRAM.....	6
1. Kode Program	7
Folder DifficultyLogic	7
Folder MainTampilan	9
Folder LeaderBoard	13
Folder settings.....	18
Folder Game.....	19
BAB 3 PENJELASAN PROGRAM DAN ANTAR KELAS.....	25
1. Penjelasan Kode Program	25
Folder MainTampilan	25
Folder DifficultyLogic	26
Folder settings.....	27
Folder LeaderBoardLogic	27
Folder MainMatchGame	29
2. Prosedur Program MatchingGame.....	30
BAB 4 PENUTUP	34

BAB 1

PENDAHULUAN

1. Latar belakang

Matching Game adalah permainan edukasi berbasis console yang dibuat dengan Java untuk melatih memori melalui tantangan mencocokkan pasangan kartu dalam waktu tercepat. Game menyediakan tiga tingkat kesulitan—Easy (4×4), Medium (6×6), dan Hard (8×8)—dan menampilkan menu utama sederhana (Start, Leaderboard, Exit). Pada saat permainan dimulai, board dibuat oleh `logikaGame.generateBoard()` yang mengacak posisi kartu secara adil menggunakan algoritma Fisher–Yates; pemain memilih dua posisi secara berurutan, dengan validasi posisi dalam batas board, kartu belum terbuka, dan larangan memilih kartu yang sama. Menang tercapai ketika semua pasangan sudah cocok dan timer yang menghitung detik dari awal sampai menang digunakan sebagai metrik skor.

Dari sisi desain, implementasi menekankan prinsip OOP: enkapsulasi diterapkan pada class `Kartu` (atribut private: `angka`, `terbuka`, `sudahCocok` dengan getter/setter), polymorphism melalui interface `difficultySelector` yang diimplementasikan oleh class `Easy`, `Medium`, dan `Hard`, serta inheritance pada sistem leaderboard di mana `leaderBoardHard` mewarisi `leaderBoard`. Abstraksi dipertahankan lewat class `logikaGame` (penyusun aturan dan alur permainan), `Tampilan` (penanganan output ke console) dan `UtilGame` (utilitas seperti `bersihkanLayar()`), sehingga kode lebih modular, reusable, dan mudah diuji.

Fitur pendukung meliputi penyimpanan skor (nama + waktu) ke leaderboard sesuai difficulty dan penampilan top 10 pemain tercepat per level dalam format tabel. Leaderboard disimpan sebagai array `User` berukuran 10 dan diurutkan naik berdasarkan waktu menggunakan insertion sort untuk menempatkan waktu tercepat di atas. Tampilan konsol menampilkan X untuk kartu tertutup dan angka untuk kartu terbuka, serta pesan umpan balik "Match Benar" atau "Match Salah", sehingga pengalaman pengguna di console menjadi jelas dan rapi.

2. Tujuan

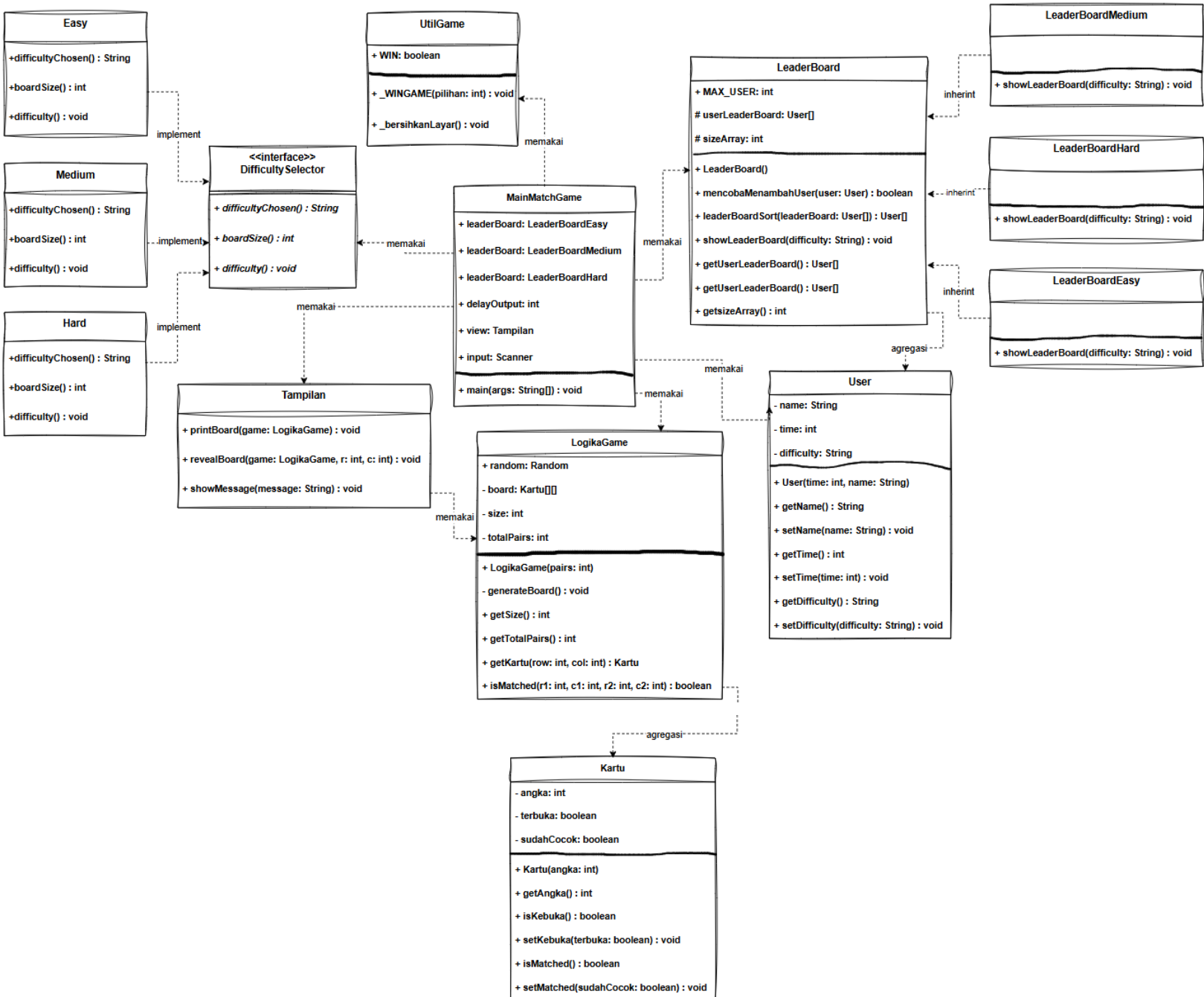
Tujuan utama dari pengembangan proyek Matching Game ini adalah untuk menerapkan secara nyata empat pilar utama Pemrograman Berbasis Objek (OOP)—yaitu encapsulation, inheritance, polymorphic, dan abstraction ke dalam sebuah program utuh berbasis Java. Melalui proyek ini, kami bertujuan untuk memahami bagaimana mengorganisasi struktur kode perangkat lunak agar menjadi lebih modular, rapi, dan mudah dirawat di masa depan, sehingga pemahaman kami tidak hanya berhenti sebagai teori hafalan di dalam kelas perkuliahan saja.

Selain pemenuhan konsep berbasis objek, proyek ini juga bertujuan untuk menguji dan melatih kemampuan logika berpikir serta penyelesaian masalah (problem solving) kami sebagai mahasiswa Informatika. Hal ini diwujudkan melalui integrasi algoritma Fisher-Yates Shuffle untuk memastikan pengacakan posisi kartu berjalan adil pada setiap sesi, serta implementasi algoritma Insertion Sort untuk mengelola sistem peringkat (leaderboard) Top 10 pemain secara dinamis dan akurat berdasarkan catatan waktu terbaik.

Terakhir, pengerjaan aplikasi dan penyusunan laporan ini ditujukan untuk memenuhi syarat penilaian Tugas Akhir pada mata kuliah Pemrograman Berbasis Objek Kelas D di Universitas Sanata Dharma. Di samping aspek akademis dan teknis tersebut, proyek bersama ini juga bertujuan untuk membangun kerja sama tim, penyamaan persepsi, serta kolaborasi antaranggota kelompok dalam merancang dan memecahkan setiap permasalahan error pemrograman secara bersama-sama.

BAB 2

SISTEM DAN DIAGRAM



1. Kode Program

Folder DifficultyLogic

- Interface DifficultySelector

```
1 package com.DifficultyLogic;
2
3 //interface yang berisi aturan method yang harus dibuat oleh class lain.
4 public interface DifficultySelector
5 {
6     abstract String difficultyChosen();
7     abstract int boardSize();
8     abstract void difficulty();
9 }
```

- Kelas Easy

```
1 package com.DifficultyLogic;
2
3 //mengimplementasikan interface difficultySelector
4 public class Easy implements DifficultySelector
5 {
6     //method untuk mode Easy: mengembalikan nama difficulty "Easy", ukuran board 2,
7     //dan menampilkan pesan bahwa tingkat kesulitannya Easy.
8     @Override
9     public String difficultyChosen()
10    {
11        return "Easy";
12    }
13
14    @Override
15    public int boardSize()
16    {
17        return 2;
18    }
19
20    @Override
21    public void difficulty()
22    {
23        System.out.println("Tingkat Kesulitan: Easy (Papan 4x4)");
24    }
25 }
```

• Kelas Medium

```
1 package com.DifficultyLogic;
2
3 public class Medium implements DifficultySelector
4 {
5     //mengembalikan nama "Medium", ukuran board 3, dan menampilkan pesan bahwa
6     //tingkat kesulitannya Medium dengan papan 6x6
7     @Override
8     public String difficultyChosen()
9     {
10         return "Medium";
11     }
12
13     @Override
14     public int boardSize()
15     {
16         return 3;
17     }
18
19     @Override
20     public void difficulty()
21     {
22         System.out.println("Tingkat Kesulitan: Medium (Papan 6x6)");
23     }
24 }
```

• Kelas Hard

```
1 package com.DifficultyLogic;
2
3 public class Hard implements DifficultySelector
4 {
5     //method untuk mode Hard: mengembalikan nama "Hard", ukuran board 4, dan
6     //menampilkan pesan bahwa tingkat kesulitannya Hard dengan papan 8x8.
7     @Override
8     public String difficultyChosen()
9     {
10         return "Hard";
11     }
12
13     @Override
14     public int boardSize()
15     {
16         return 4;
17     }
18
19     @Override
20     public void difficulty()
21     {
22         System.out.println("Tingkat Kesulitan: Hard (Papan 8x8)");
23     }
24 }
```


Folder MainTampilan

- **Kelas Kartu**

```
1 package com.MainTampilan;
2
3 //Objek Kartu untuk pemetaan dan pencocokan
4 public class Kartu
5 {
6     private int angka;
7     private boolean terbuka;
8     private boolean sudahCocok;
9
10    //constructor untuk membuat objek Kartu dengan nilai angka tertentu.
11    public Kartu(int angka)
12    {
13        this.angka = angka;
14        this.terbuka = false;
15        this.sudahCocok = false;
16    }
17    public int getAngka()
18    {
19        return angka;
20    }
21
22    public boolean isKebuka()
23    {
24        return terbuka;
25    }
26
27    public void setKebuka(boolean terbuka)
28    {
29        this.terbuka = terbuka;
30    }
31
32    public boolean isMatched()
33    {
34        return sudahCocok;
35    }
36
37    public void setMatched(boolean sudahCocok)
38    {
39        this.sudahCocok = sudahCocok;
40    }
41 }
42
```

- **Kelas Tampilan**

```
1 package com.MainTampilan;
2
3 public class Tampilan
4 {
5     //untuk menampilkan board permainan ke jendela output
6     public void printBoard(LogikaGame game)
7     {
8         System.out.println(x: "Board game");
9         for (int i = 0; i < game.getSize(); i++) {
10             for (int j = 0; j < game.getSize(); j++) {
11                 Kartu k = game.getKartu(row: i, col: j);
12
13                 // kalau terbuka atau udah match, tampilin angkanya
14                 if (k.isKebuka() || k.isMatched()) {
15                     System.out.print(k.getAngka() + " ");
16                 } else {
17                     System.out.print(s: "X ");
18                 }
19             }
20             System.out.println();
21         }
22     }
23
24     public void revealBoard(LogikaGame game, int r, int c)
25     {
26         // buka kartunya
27         game.getKartu(row: r, col: c).setKebuka(terbuka: true);
28         printBoard(game);
29     }
30
31     public void showMessage(String message)
32     {
33         System.out.println(x: message);
34     }
35 }
```

- **Kelas LogikaGame**

```
1 package com.MainTampilan;
2
3 import java.util.Random;
4
5 //untuk menyimpan data utama permainan, angka kartu, apakah kartu
6 //sudah dibuka, dan apakah kartu sudah cocok
7 public class LogikaGame
8 {
9     public Random random = new Random();
10    private Kartu[][] board;
11    private int size;
12    private int totalPairs;
13
14    //Mempetakan Barisan Peta MatchGame
15    public LogikaGame(int pairs)
16    {
17        this.size = pairs * 2;
18        this.totalPairs = (size*size)/2;
19        //kalau sebelumnya pakai map tile dan map revealed,
20        //file kartu.java nyimpan itu biar ada encapsulation
21        this.board = new Kartu[size][size];
22        generateBoard();
23    }
24
25    //Membuat Barisan Angka untuk pemetaan
26    private void generateBoard()
27    {
28        int[] randomNumbers = new int[size * size];
29
30        // bikin sepasang angka
31        for (int i = 0; i < totalPairs; i++) {
32            randomNumbers[i * 2] = i;
33            randomNumbers[i * 2 + 1] = i;
34        }
35
36        //acak angka
37        for (int i = randomNumbers.length - 1; i > 0; i--)
```

```

39         int j = random.nextInt(i + 1);
40         int temp = randomNumbers[i];
41         randomNumbers[i] = randomNumbers[j];
42         randomNumbers[j] = temp;
43     }
44
45     //isi ke array objek
46     for (int i = 0; i < size; i++)
47     {
48         for(int j = 0; j < size; j++)
49         {
50             board[i][j] = new Kartu(randomNumbers[i * size + j]);
51         }
52     }
53 }
54
55 public int getSize()
56 {
57     return size;
58 }
59
60 public int getTotalPairs()
61 {
62     return totalPairs;
63 }
64
65 //pindahin ke kartu.jawa e biar rapih dan bisa diakses sama mainmatchgame
66 public Kartu getKartu(int row, int col)
67 {
68     return board[row][col];
69 }
70
71 public boolean isMatched(int r1, int c1, int r2, int c2)
72 {
73     // cek kalau angkanya sama tapi posisinya beda
74     if (r1 == r2 && c1 == c2) return false;
75     return board[r1][c1].getAngka() == board[r2][c2].getAngka();

```

Folder LeaderBoard

- Kelas LeaderBoard

```
1 package com.LeaderBoardLogic;
2
3 //class leaderBoard untuk menyimpan dan mengurutkan top 10 user berdasarkan waktu tercepat
4
5 public class LeaderBoard
6 {
7     public final int MAX_USER = 10;
8     protected User[] userLeaderBoard;
9     protected int sizeArray = 0;
10
11     public LeaderBoard()
12     {
13         this.userLeaderBoard = new User[MAX_USER];
14     }
15
16     public boolean mencobaMenambahUser(User user)
17     {
18         if(sizeArray < MAX_USER)
19         {
20             userLeaderBoard[sizeArray] = user;
21             sizeArray++;
22             leaderBoardSort(leaderBoard: userLeaderBoard);
23             return true;
24         }
25
26         if(user.getTime() < userLeaderBoard[sizeArray - 1].getTime())
27         {
28             userLeaderBoard[sizeArray - 1] = user;
29             leaderBoardSort(leaderBoard: userLeaderBoard);
30             return true;
31         }
32
33         return false;
34     }
35
36     //Penyusunan LeaderBoard sesuai User time clear dan tingkat kesusahan
37     public User[] leaderBoardSort(User[] leaderBoard)
38     {
```

```

38     for(int i = 1; i < sizeArray; i++)
39     {
40         User userTime = leaderBoard[i];
41         double key = leaderBoard[i].getTime();
42         int j = i - 1;
43
44         while(j >= 0 && leaderBoard[j].getTime() > key)
45         {
46             leaderBoard[j + 1] = leaderBoard[j];
47             j--;
48         }
49         leaderBoard[j + 1] = userTime;
50     }
51
52     return leaderBoard;
53 }
54
55
56 public void showLeaderBoard(String difficulty)
57 {
58     if(sizeArray == 0)
59     {
60         System.out.println(x: "Belum Ada User");
61         return;
62     }
63
64     System.out.println("Difficulty : " + difficulty);
65     System.out.printf(format: ""
66         %-5s | %-7s | %s
67         """,
68         args: "No",
69         args: "Nama",
70         args: "Time");
71     System.out.println(x: "=====");
72     for(int i = 0; i < sizeArray; i++)
73     {
74         String nama = userLeaderBoard[i].getName();
75         double time = userLeaderBoard[i].getTime();
76
77         System.out.printf(format: ""
78             %-5d | %-7s | %.0f
79             """,
80             (i + 1),
81             args: nama,
82             args: time);
83     }
84     System.out.println(x: "=====");
85
86
87 public User[] getUserLeaderBoard()
88 {
89     return userLeaderBoard;
90 }
91
92 public int getsizeArray()
93 {
94     return sizeArray;
95 }

```

- Kelas LeaderBoardEasy

```
1 package com.LeaderBoardLogic;
2
3 //class leaderBoardEasy yang mewarisi class leaderBoard
4 //semua fungsi dan atribut dari leaderBoard dipakai juga di sini
5 public class LeaderBoardEasy extends LeaderBoard
6 {
7
8     @Override
9     public void showLeaderBoard(String difficulty)
10    {
11        if(sizeArray == 0)
12        {
13            System.out.println("Belum Ada User");
14            return;
15        }
16
17        System.out.println("Difficulty : " + difficulty + ", Board 4x4");
18        System.out.printf("
19            %-5s | %-7s | %s
20            """,
21            "No",
22            "Nama",
23            "Time");
24        System.out.println("=====");
25        for(int i = 0; i < sizeArray; i++)
26        {
27            String nama = userLeaderBoard[i].getName();
28            double time = userLeaderBoard[i].getTime();
29            System.out.printf("
30                %-5d | %-7s | %.0f
31                """,
32                (i + 1),
33                nama,
34                time);
35        }
36        System.out.println("=====");
37    }
38 }
```

- Kelas LeaderBoardMedium

```
1 package com.LeaderBoardLogic;
2
3 //class leaderBoardMedium yang mewarisi class leaderBoard
4 //class ini juga memakai semua atribut dan method dari leaderBoard
5 public class LeaderBoardMedium extends LeaderBoard
6 {
7     @Override
8     public void showLeaderBoard(String difficulty)
9     {
10         if(sizeArray == 0)
11         {
12             System.out.println("Belum Ada User");
13             return;
14         }
15
16         System.out.println("Difficulty : " + difficulty + ", Board 6x6");
17         System.out.printf("""
18             %-5s | %-7s | %s
19             """,
20             "No",
21             "Nama",
22             "Time");
23         System.out.println("=====");
24         for(int i = 0; i < sizeArray; i++)
25         {
26             String nama = userLeaderBoard[i].getName();
27             double time = userLeaderBoard[i].getTime();
28             System.out.printf("""
29                 %-5d | %-7s | %.0f
30                 """,
31                 (i + 1),
32                 nama,
33                 time);
34         }
35         System.out.println("=====");
36     }
37 }
```


- Kelas LeaderBoardHard

```
1 package com.LeaderBoardLogic;
2
3 //class leaderBoardHard yang mewarisi class leaderBoard
4 //class ini juga memakai semua atribut dan method dari leaderBoard
5 public class LeaderBoardHard extends LeaderBoard
6 {
7     @Override
8     public void showLeaderBoard(String difficulty)
9     {
10         if(sizeArray == 0)
11         {
12             System.out.println("Belum Ada User");
13             return;
14         }
15
16         System.out.println("Difficulty : " + difficulty + ", Board 8x8");
17         System.out.printf("""
18             %-5s | %-7s | %s
19             """,
20             "No",
21             "Nama",
22             "Time");
23         System.out.println("=====");
24         for(int i = 0; i < sizeArray; i++)
25         {
26             String nama = userLeaderBoard[i].getName();
27             double time = userLeaderBoard[i].getTime();
28             System.out.printf("""
29                 %-5d | %-7s | %.0f
30                 """,
31                 (i + 1),
32                 nama,
33                 time);
34         }
35         System.out.println("=====");
36     }
37 }
```

Folder settings

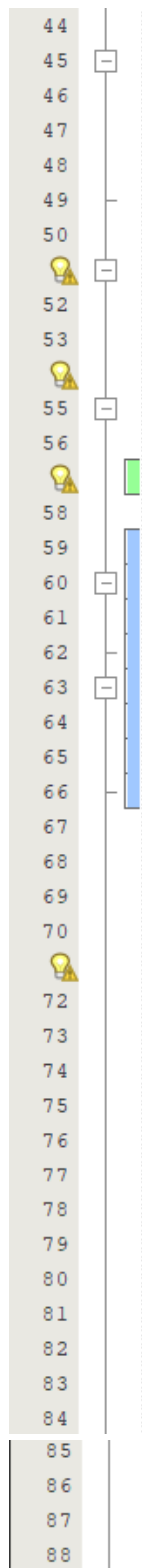
- Kelas UtilGame

```
1 package com.settings;
2
3 //Class berisi settingan tampilan atau game
4 public class UtilGame
5 {
6     //Kemudahan debugging
7     public static boolean WIN = false;
8
9     //Easter Egg
10    public static void WINGAME(int pilihan)
11    {
12        switch (pilihan)
13        {
14            case 67:
15                System.out.println("67");
16                try {Thread.sleep(1500);} catch (Exception e) {}
17                break;
18            case 100:
19                System.out.println("Halo");
20                try {Thread.sleep(1500);} catch (Exception e) {}
21                break;
22            case -1:
23                System.out.println("Rawr");
24                try {Thread.sleep(1500);} catch (Exception e) {}
25                break;
26            case 777:
27                WIN = true;
28                try {Thread.sleep(1500);} catch (Exception e) {}
29                break;
30        }
31    }
32
33    //Metode membersihkan layar
34    public static void bersihkanLayar()
35    {
36        int Baris = 40;
37        for(int i = 0; i < Baris; i++)
38        {
39            System.out.println();
40        }
41    }
42 }
```

Folder Game

- Kelas MainMatchGame

```
1 package com.Game;
2
3 import java.util.Scanner;
4
5 //Import File yang diperlukan
6 import com.DifficultyLogic.*;
7 import com.LeaderBoardLogic.*;
8 import com.MainTampilan.*;
9 import com.settings.UtilGame;
10
11 //Class utama yang menjalankan game
12 public class MainMatchGame
13 {
14     //Bikin objek untuk masing-masing class yang diperlukan
15     static LeaderBoard leaderBoardEasy = new LeaderBoardEasy();
16     static LeaderBoard leaderBoardMedium = new LeaderBoardMedium();
17     static LeaderBoard leaderBoardHard = new LeaderBoardHard();
18     static int delayOutput = 1500;
19     static Tampilan view = new Tampilan();
20     static Scanner input = new Scanner(System.in);
21
22     //Main Method yang menjalankan game
23     public static void main(String[] args)
24     {
25         boolean isMenuBerjalan = true;
26
27         // Loop utama untuk menu
28         while (isMenuBerjalan)
29         {
30             UtilGame.bersihkanLayar();
31             int pilihan = -1;
32             view.showMessageDialog("Matching Game : Kelompok Romusha");
33             view.showMessageDialog("1. Start");
34             view.showMessageDialog("2. Leaderboard");
35             view.showMessageDialog("3. Exit");
36             view.showMessageDialog("=====");
37             System.out.print("Pilih: ");
38
39             // Validasi input untuk menu utama
40             try
41             {
42                 pilihan = input.nextInt();
43             }
44             catch (Exception e)
```



```
catch (Exception e)
{
    view.showMessageDialog("Input harus angka");
    input.nextLine();
    continue;
}

try {Thread.sleep(delayOutput);} catch (Exception e){}

//Pilihan main main game, lihat leaderboard, atau keluar
if (pilihan == 1)
{
    // Logika milih level
    int level = -1;
    view.showMessageDialog("Pilih: 1. Easy | 2. Medium | 3. Hard");
    try
    {
        level = input.nextInt();
    }
    catch(Exception e){
        input.nextLine();
        continue;
    }

    DifficultySelector diff;

    //Pilih level tingkat kesusahan
    switch (level)
    {
        case 1:
            diff = new Easy();
            break;
        case 2:
            diff = new Medium();
            break;
        case 3:
            diff = new Hard();
            break;
        default:
            view.showMessageDialog("Salah Pilih, Coba Lagi");
            continue;
    }

    //Dapatkan ukuran papan dari level yang dipilih
    String diffSelect = diff.difficultyChosen();
```

```

89     diff.difficulty();
90     int ukuranPapan = diff.boardSize();
91
92     double waktumulai = System.nanoTime();
93
94     //Pemetaan Game Matching Game
95     LogikaGame game = new LogikaGame(ukuranPapan);
96
97     int matchedPairs = 0;
98
99     // Loop utama untuk permainan, akan terus berjalan sampai
100    // semua pasangan kartu ditemukan
101    while (matchedPairs < game.getTotalPairs())
102    {
103        UtilGame.bersihkanLayar();
104        view.printBoard(game);
105
106        try
107        {
108            // Pilih kartu pertama
109            view.showMessage("Pilih kartu pertama (baris lalu kolom): ");
110            System.out.print("Baris : ");
111            int Baris1 = input.nextInt();
112            input.nextLine();
113            System.out.print("Kolom : ");
114            int Kolom1 = input.nextInt();
115            input.nextLine();
116
117            //Validasi input untuk kartu pertama
118            if (Baris1 < 0 || Baris1 >= game.getSize() || Kolom1 < 0 || Kolom1 >= game.getSize())
119            {
120                view.showMessage("Posisi kartu 1 di luar batas!");
121                Thread.sleep(delayOutput);
122                continue;
123            }
124
125            if (game.getKartu(Baris1, Kolom1).isMatched() || game.getKartu(Baris1, Kolom1).isKebuka())
126            {
127                view.showMessage("Kartu itu udah kebuka!");
128                Thread.sleep(delayOutput);
129                continue;
130            }
131
132            UtilGame.bersihkanLayar();
133            view.revealBoard(game, Baris1, Kolom1);
134
135            // Pilih kartu kedua
136            view.showMessage("Pilih kartu kedua (baris lalu kolom): ");
137            System.out.print("Baris : ");
138            int Baris2 = input.nextInt();
139            input.nextLine();
140            System.out.print("Kolom : ");
141            int Kolom2 = input.nextInt();
142            input.nextLine();

```

```

143
144
145 // Secret Feature
146 UtilGame.WINGAME(Kolom2);
147
148 //Kemudahan debugging
149 if(UtilGame.WIN)
150 {
151     UtilGame.WIN = false;
152     matchedPairs = game.getTotalPairs();
153     game.getKartu(Baris1, Kolom1).setKebuka(false);
154     continue;
155 }
156
157 //Validasi input untuk kartu kedua
158 if (Baris2 < 0 || Baris2 >= game.getSize() || Kolom2 < 0 || Kolom2 >= game.getSize())
159 {
160     view.showMessage("Posisi kartu 2 di luar batas!");
161     Thread.sleep(delayOutput);
162     game.getKartu(Baris1, Kolom1).setKebuka(false); // tutup lagi kartu 1
163     continue;
164 }
165
166 if ((Baris1 == Baris2 && Kolom1 == Kolom2) || game.getKartu(Baris2, Kolom2).isMatched())
167 {
168     view.showMessage("Gak boleh pilih kartu yang sama!");
169     Thread.sleep(delayOutput);
170     game.getKartu(Baris1, Kolom1).setKebuka(false);
171     continue;
172 }
173
174 UtilGame.bersihkanLayar();
175 view.revealBoard(game, Baris2, Kolom2);
176
177 // Cek Match
178 if (game.isMatched(Baris1, Kolom1, Baris2, Kolom2))
179 {
180     view.showMessage("Match Benar");
181     Thread.sleep(delayOutput);
182     game.getKartu(Baris1, Kolom1).setMatched(true);
183     game.getKartu(Baris2, Kolom2).setMatched(true);
184     matchedPairs++;
185 }
186 else
187 {
188     view.showMessage("Match Salah");
189     Thread.sleep(delayOutput);
190     game.getKartu(Baris1, Kolom1).setKebuka(false);
191     game.getKartu(Baris2, Kolom2).setKebuka(false);
192 }
193
194 catch (Exception e)
195 {
196     view.showMessage("Input harus angka");
197     try {Thread.sleep(delayOutput);} catch (Exception w) {}
198     input.nextLine(); // clear buffer
199 }

```

```

200
201 // Permainan selesai, tampilkan papan dan waktu
202 view.printBoard(game);
203 view.showMessage("SELAMAT! Kamu menang!");
204
205 double waktuakhir = System.nanoTime();
206 int time = (int) ((waktuakhir - waktumulai) / 1000000000.0); //siapa tau
207
208 view.showMessage("Nama Inisialmu : ");
209 String nama = input.nextLine();
210
211 if (nama.isEmpty())
212 {
213     nama = "Player";
214 }
215
216 //Pembuatan objek user untuk menyimpan skor dan nama pemain, lalu disimpan
217 //ke leaderboard sesuai tingkat kesusahan yang dipilih
218 User player = new User(time, nama);
219
220 switch (diffSelect)
221 {
222     case "Easy":
223         player.setDifficulty(diffSelect);
224         leaderboardEasy.mencobaMenambahUser(player);
225         break;
226     case "Medium":
227         player.setDifficulty(diffSelect);
228         leaderboardMedium.mencobaMenambahUser(player);
229         break;
230     case "Hard":
231         player.setDifficulty(diffSelect);
232         leaderboardHard.mencobaMenambahUser(player);
233         break;
234 }
235
236 // Tampilkan skor yang disimpan
237 view.showMessage("Skor disimpan ,waktu: " + player.getTime() + " detik, Kesusahan: " + player.getDifficulty());
238 try {Thread.sleep(delayOutput);} catch (Exception e) {}
239
240 //Pilihan untuk melihat leaderboard
241 else if (pilihan == 2)
242 {
243     UtilGame.bersihkanLayar();
244
245     // Menu untuk memilih tingkat kesusahan leaderboard yang ingin dilihat
246     view.showMessage("Silahkan Masukkan Kategori Tingkat Kesulitan LeaderBoard");
247     view.showMessage("1. Easy");
248     view.showMessage("2. Medium");
249     view.showMessage("3. Hard");
250     System.out.print("Pilihan : ");
251     pilihan = -1;
252     try
253     {
254         pilihan = input.nextInt();
255         input.nextLine();
256         Thread.sleep(delayOutput);
257     }
258     catch (Exception e) {input.nextLine();}
259
260 // Tampilkan leaderboard sesuai pilihan tingkat kesusahan
261 switch (pilihan)
262 {
263     case 1:
264         leaderboardEasy.showLeaderBoard("Easy");
265         break;
266     case 2:
267         leaderboardMedium.showLeaderBoard("Medium");
268         break;
269     case 3:
270         leaderboardHard.showLeaderBoard("Hard");
271         break;
272     default:
273         view.showMessage("Something's Wrong, try again");
274         try {Thread.sleep(delayOutput);} catch (Exception e){}
275         break;
276 }

```

```
277         view.showMessageDialog("Press Enter to go back");
278         input.nextLine(); // nersih enter
279     }
280     //Pilihan untuk keluar dari game
281     else if (pilihan == 3)
282     {
283         view.showMessageDialog("Keluar Game...");
284         try {Thread.sleep(delayOutput);} catch (Exception e) {}
285         isMenuBerjalan = false; // Bikin loop menu berhenti
286     }
287 }
288 input.close();
289 }
290 }
```


BAB 3

PENJELASAN PROGRAM DAN ANTAR KELAS

1. Penjelasan Kode Program

Folder MainTampilan

1. Kelas Kartu

Kelas ini adalah kelas untuk membuat objek-objek untuk pemetaan map match game dimana kelas objek hanya untuk melakukan operasi seperti isMatched() dalam kelas LogikaGame untuk melakukan logic matching angka

2. Kelas LogikaGame

Kelas LogikaGame bertujuan untuk melakukan pemetaan map match game dimana membuat sebuah board yang berisi angka-angka serta mengacak ulang board tersebut melewati generateBoard() dimana acak tersebut menggunakan java.util.Random untuk melakukan pengacakan

3. Kelas Tampilan

Kelas Tampilan adalah dimana user melihat hasil board tersebut menggunakan printBoard() dimana saat user melakukan permainan akan membuat sebuah kotak persegi dalam terminal yang berisi tanda X sehingga tujuan user adalah untuk menginput sebuah koordinat 2D array untuk membandingkan semua angka yang sama dalam board tersebut dan cara pengungkapan adalah dengan method revealBoard() dimana akan memperlihatkan angka-angka yang sama dalam board serta menggunakan method showMessage() untuk melakukan output kalimat dimana sebagai pengganti system.out.print() untuk method tertentu

Folder DifficultyLogic

1. Interface DifficultySelector

Kelas ini bertujuan untuk membuat implementasi ukuran board dari pilihan user dengan berbagai tingkat kesusahan yaitu east, medium, dan hard. Dimana membuat board berdasarkan berapa besar ukuran board berdasarkan tingkat kesusahan

2. Kelas Easy

Kelas yang mengimplementasi DifficultySelector dimana untuk membuat ukuran board 4 x 4

3. Kelas Medium

Kelas yang mengimplementasi DifficultySelector dimana untuk membuat ukuran board 6 x 6

4. Kelas Hard

Kelas yang mengimplementasi DifficultySelector dimana untuk membuat ukuran board 8 x 8

Folder settings

1. Kelas UtilGame

Kelas ini bertujuan untuk membuat tampilan settingan seperti pembersihLayar() serta gameplay tambahan seperti input rahasia lewat method WINGAME()

Folder LeaderBoardLogic

1. Kelas LeaderBoard

Kelas ini bertujuan untuk membuat sebuah leaderboard untuk melacak waktu selesai sebuah user dan juga melakukan perurutan dari waktu terpendek dan dimana leaderboard tersebut hanya bisa menampung 10 User. LeaderBoard ini digunakan untuk mewariskan atribut dan method-method ke leaderboard yang tergantung dengan difficulty masing-masing

2. Kelas LeaderBoardEasy

Kelas yang mewariskan atribut-atribut dari kelas LeaderBoard dengan mengubah method tampilan showLeaderBoard() dengan ukuran board 4x4

3. Kelas LeaderBoardMedium

Kelas yang mewariskan atribut-atribut dari kelas LeaderBoard dengan mengubah method tampilan showLeaderBoard() dengan ukuran board 6x6

4. Kelas LeaderBoardHard

Kelas yang mewariskan atribut-atribut dari kelas LeaderBoard dengan mengubah method tampilan showLeaderBoard() dengan ukuran board 8x8

Pengecekan Kapasitas Leaderboard:

Melalui metode mencobaMenambahUser(User user):

Jika jumlah skor tersimpan (sizeArray) kurang dari kapasitas maksimum (MAX_USER = 10), maka objek User langsung dimasukkan ke index ke sizeArray, dan sizeArray dinaikkan sebesar 1. Setelah itu dilakukan pengurutan.

Jika kalau leaderboard sudah penuh (berisi max :10 user), program membandingkan waktu user baru dengan waktu user di peringkat terakhir (terlambat) yaitu index ke 9, Jika waktu user baru lebih cepat (lebih kecil; yang berarti terbaru), maka user peringkat ke 10 ditimpa dengan user baru tersebut, lalu dilakukan pengurutan ulang.

Algoritma Sorting (Insertion Sort):

Pengurutan dilakukan secara menaik oleh metode leaderBoardSort() menggunakan algoritma Insertion Sort

Tampilan Leaderboard:

Pemain dapat melihat daftar 10 skor terbaik per kategori melalui menu utama. Program akan memanggil metode showLeaderBoard() yang mencetak nama dan waktu penyelesaian user secara rapi dan terformat.

Folder MainMatchGame

1. Kelas MainMatchGame

Kelas ini adalah kelas utama dan untuk menjalankan main program. Kelas utama ini adalah untuk membuat tampilan untuk memasuki menu utama permainan, dan mengatur jalannya seluruh alur loop game. Dalam main program ini, User mendapati dua pilihan saat menjalani program yaitu Start dan LeaderBoard. Saat memilih Start, User akan ditampilkan 3 pilihan tingkat kesusahan untuk melakukan permainan dimana tingkat kesusahan tersebut akan memperbesar ukuran board match game. User tersebut akan memainkan permainan tersebut seperti biasa dimana harus mencocokkan angka yang sudah tersebar di board dengan angka lainnya. Permainan tersebut akan selesai jika User berhasil mencocok semua angka yang sama. Setelah selesai, User akan ditampilkan sebuah input dimana akan dimasukkan nama user dan hasil yang diinputkan akan ditampilkan beserta waktu selesai kedalam leaderboard masing-masing.

2. Prosedur Program MatchingGame

- 1) User akan ditampilkan Sebuah main menu dengan pilihan 1. Start, 2. LeaderBoard, dan 3. Exit

```
Matching Game : Kelompok Romusha
1. Start
2. Leaderboard
3. Exit
=====
Pilih:
```

- 2) Untuk memulaikan permainan, User memilih/mengketik 1 untuk menampilkan pilihan tingkat kesusahan dimana user dengan bebas memilih 3 pilihan yaitu easy, medium, dan hard

```
Matching Game : Kelompok Romusha
1. Start
2. Leaderboard
3. Exit
=====
Pilih: 1
Pilih: 1. Easy | 2. Medium | 3. Hard
||
```

- 3) Ketika sudah memilih tingkat kesusahan User akan diberikan sebuah board dengan isinya X yang tersusun rapi pada permainan tersebut dan user akan ditampilkan pilihan Baris dan Kolom.

```
Board game
X X X X
X X X X
X X X X
X X X X
Pilih kartu pertama (baris lalu kolom):
Baris : 2
Kolom :
```

- 4) User akan memilih sebuah Baris dan Kolom dua kali dengan pilihan baris dan kolom yang kedua tidak boleh sama dengan baris dan kolom yang kesatu, User akan menampilkan isi angka dari dua baris dan kolom tersebut

```
Board game
0 X X X
X X X X
X 6 X X
X X X X
Match Salah
```

- 5) Dari gambar sebelumnya. Game akan menampilkan dua angka tersebut sesuai dengan baris dan kolom dari inputan tersebut dan Game akan memproses hasil muncul dua angka tersebut dan mencoba apakah angka tersebut sama atau bukan
- 6) Jika tidak sama, Game akan membalikan kedua angka tersebut dalam kondisi sebelumnya saat menjadi X dan balik lagi di pemilihan baris dan kolom pertama

```
Board game
X X X X
X X X X
X X X X
X X X X
Pilih kartu pertama (baris lalu kolom):
Baris : |
```

- 7) Dari hasil tersebut, User harus mencocokkan semua angka yang sama yang ada di board dan memenangkan Game

```
Board game
X X X 6
X X X X
X 6 X X
X X X X
Match Benar
```

- 8) Jika User sudah mencocokkan semua angka yang ada di board akan ditampilkan sebuah tampilan kemenangan dimana user harus input inisial/nama

```
Board game
X X X 6
X X X X
X 6 X X
X X X X
SELAMAT! Kamu menang!
Nama Inisialmu :
||
```

*penggunaan insta win untuk kemudahan tugas

- 9) Setelah User menginputkan nama tersebut. User akan ditampilkan waktu penyelesaian dan tingkat kesusahan yang User barusan mainkan dan kembali lagi ke main menu game

```
Board game
X X X 6
X X X X
X 6 X X
X X X X
SELAMAT! Kamu menang!
Nama Inisialmu :
Contoh
Skor disimpan ,waktu: 454 detik, Kesusahan: Easy
```

- 10) Hasil menang tersebut akan disimpan ke leaderboard dimana disusun sesuai dengan waktu cepat selesai game dan diisi sesuai tingkat kesusahan masing-masing

```
Matching Game : Kelompok Romusha
1. Start
2. Leaderboard
3. Exit
=====
Pilih: 2
```

```
Silahkan Kategori Tingkat Kesulitan LeaderBoard
1. Easy
2. Medium
3. Hard
Pilihan : 1
Difficulty : Easy, Board 4x4
No      | Nama      | Time
=====
1       | Contoh   | 454
=====
Press Enter to go back
```


11) Ketika User ingin keluar dari game, User disilahkan input 3 untuk keluar dari game

```
Matching Game : Kelompok Romusha
```

```
1. Start
```

```
2. Leaderboard
```

```
3. Exit
```

```
=====
```

```
Pilih: 3
```

```
Keluar Game...
```

```
-----
```

```
BUILD SUCCESS
```

```
-----
```

```
Total time: 18:28 min
```

```
Finished at: 2026-05-30T23:43:26+07:00
```

```
-----
```

```
||
```

BAB 4

PENUTUP

Berdasarkan seluruh proses perencanaan, implementasi kode program, hingga penyusunan laporan untuk proyek Matching Game ini, dapat disimpulkan bahwa penerapan empat pilar utama OOP bukan sekadar teori hafalan di dalam kelas. Melalui struktur game ini, kami berhasil mengimplementasikan Encapsulation pada status data kelas Kartu, Inheritance pada hierarki kelas LeaderBoard, Polymorphism melalui pemanfaatan interface DifficultySelector, serta Abstraction untuk memisahkan fungsi visual di kelas Tampilan dan mesin mekanis di kelas LogikaGame. Pendekatan berbasis objek ini terbukti membuat struktur kode program kelompok kami menjadi jauh lebih rapi, modular, dan mudah untuk dikembangkan di kemudian hari.

Selain keberhasilan dalam menyusun arsitektur objek, proyek ini juga membuktikan bahwa integrasi algoritma dalam pemrograman dapat berjalan dengan sangat efektif. Penggunaan algoritma Fisher-Yates Shuffle sukses menghasilkan acakan posisi kartu yang adil pada setiap sesi permainan baru, sementara penerapan algoritma Insertion Sort pada sistem leaderboard berhasil menyortir waktu penyelesaian pemain secara real-time untuk mempertahankan akurasi data posisi sepuluh besar. Proses pemecahan logika ini secara tidak langsung telah mengasah ketajaman berpikir kami sebagai mahasiswa Informatika, terutama dalam memanipulasi koordinat matriks array dua dimensi serta menangani validasi status kartu yang dinamis.

Secara keseluruhan, proyek Tugas Akhir ini tidak hanya berhasil menghasilkan sebuah aplikasi game edukasi pelatih memori yang fungsional, tetapi juga memberikan pengalaman praktis yang sangat berharga bagi kami. Kami belajar banyak tentang bagaimana mengelola alur permainan yang interaktif secara tim, menyatukan pemikiran dalam memecahkan masalah coding, serta memperkuat pemahaman mendasar mengenai implementasi bahasa pemrograman Java dalam sebuah studi kasus nyata.